

No MMU.
No debugger.
No excuses.

The ESP32 cannot protect one program from another — its MMU translates flash addresses and nothing else. **nat-os recovers that guarantee in software**, running applications inside a bytecode interpreter that bounds-checks every load and store. A program written for no purpose but to escape its arena is part of the test suite. It faults at offset 256 of a 256-byte arena, and its neighbours keep running.

This is the complete account, built from the source, the commit history and twenty-eight engineering reports — and it records the failures with the same care as the successes, because **that is where the transferable knowledge is**. A touch axis inverted for three months behind a calibration that could only ever return one answer. Three peripherals whose registers read back perfectly while the hardware sat dead.

- Preemptive scheduling
- Bytecode VM
- Software isolation
- ILI9341 + DMA
- XPT2046 touch
- Flash persistence
- microSD
- SAR ADC
- Bit-banged I2C
- LEDC audio
- 802.11 receive

372

PAGES

35

OPCODES

0

ESCAPES

12

STANDING RULES

Developed and verified on the ESP32-2432S028R, the board sold as the “Cheap Yellow Display”. The screens on the front are reconstructions rendered from the kernel's own font, icon bitmaps, map data and RGB565 colour constants — not photographs of the board.

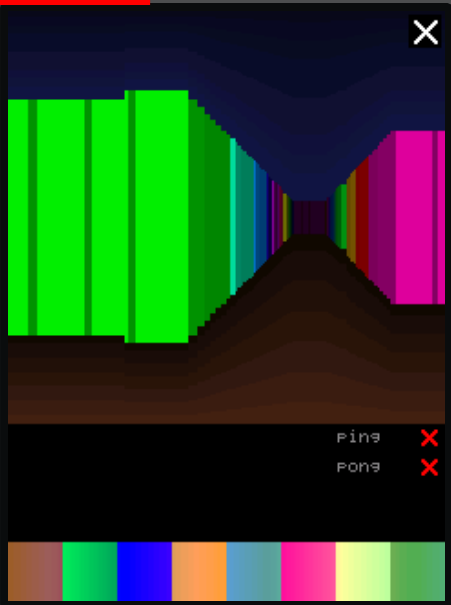
Source, reports and this book: github.com/nFerragut88/nat-os · MIT licence

nat-os An OS From Scratch for the ESP32 USED MEDIAS

nat-os

AN OPERATING SYSTEM WRITTEN FROM SCRATCH
FOR THE ESP32

3D VIEW



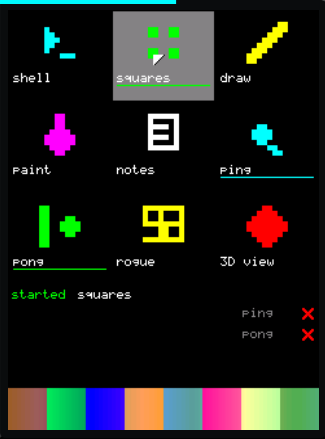
WHAT IS IN HERE

Preemptive scheduling with ageing.
A bytecode VM, 35 opcodes.
Software memory isolation.
ILI9341 + DMA, 43 ms full screen.
Resistive touch, calibrated.
Flash, microSD, ADC, I²C, audio.
802.11 receive, through a blob
in the other calling convention.

AND EVERY DEFECT THAT GOT THERE FIRST

31 chapters · 7 appendices
28 engineering reports

LAUNCHER



SHELL



NOTES



No ESP-IDF · No FreeRTOS · No C library · 37,248 bytes
Every instruction from the image entry point onward is project code